

Compiler Grammar Specification

Three Address Code (TAC) & Assembly Generator

1. High-Level Structure

```
start          : program
program        : program unit | unit
unit           : var_declaration | func_definition
```

2. Function & Variable Declarations

```
func_definition : type_specifier ID LPAREN parameter_list RPAREN
                 compound_statement
                 | type_specifier ID LPAREN RPAREN
                 compound_statement

parameter_list  : parameter_list COMMA type_specifier ID
                 | type_specifier ID
                 | type_specifier

var_declaration : type_specifier declaration_list SEMICOLON
type_specifier  : INT | FLOAT | VOID
declaration_list : ID
                 | ID LTHIRD CONST_INT RTHIRD
                 | declaration_list COMMA ID
```

3. Statements & Control Flow

```
compound_stmt  : LCURL statements RCURL | LCURL RCURL
statements     : statement | statements statement
statement      : var_declaration
                | expression_statement
                | compound_statement
                | IF LPAREN expression RPAREN statement
                | IF LPAREN expression RPAREN statement ELSE
statement      : WHILE LPAREN expression RPAREN statement
statement      : FOR LPAREN expr_stmt expr_stmt expression RPAREN
statement      : PRINTLN LPAREN ID RPAREN SEMICOLON
statement      : RETURN expression SEMICOLON
```

4. Expressions & Operators

```
expression     : logic_expression
                | variable ASSIGNOP logic_expression

logic_expr     : rel_expression
                | rel_expression LOGICOP rel_expression

rel_expr       : simple_expression
                | simple_expression RELOP simple_expression

simple_expr     : term
                | simple_expression ADDOP term

term           : unary_expression
                | term MULOP unary_expression

unary_expr     : factor
                | ADDOP unary_expression
```

| **NOT** unary_expression

5. Factors & Function Calls

```
factor          : variable
                  | CONST_INT | CONST_FLOAT
                  | LPAREN expression RPAREN
                  | ID LPAREN argument_list RPAREN
                  | variable INCOP | DECOP

variable       : ID | ID LTHIRD expression RTHIRD
argument_list : arguments | /* empty */
arguments     : logic_expression | arguments COMMA
logic_expression
```